1. Langage choisi : C

Le projet est écrit en C.

2. Choix algorithmiques

Choix 1 — Stockage des colis

Problème	Ranger un nombre de colis inconnu à l'avance (jusqu'à 100 000) en gardant un ajout rapide.
Solution	Tableau dynamique de Colis rangés à la suite en mémoire. Quand il est plein, sa capacité double (realloc).
Temps	O(n).
Espace	O(n)
Alternative	Liste chaînée mais un malloc par colis, accès à un élément en O(n), à chaque ajout la complexité serait de grandeur $O(n^2)$.
Justification	Le doublement garde l'ajout rapide, les colis contigus se parcourent et se copient vite, et la position d'un colis ne change jamais.

Choix 2 — Recherche d'un colis par identifiant

Problème	Retrouver un colis à partir de son id, et refuser un id déjà utilisé à chaque ajout, sans parcourir toute la collection.
Solution	Table de hachage indexée. Si une case est prise, on essaie la suivante. La table est doublée et reconstruite dès qu'elle est remplie à plus de 50 %.
Temps	O(n) dans le pire cas 0(1) dans le meilleur cas
Espace	O(n)
Alternative	Parcours du tableau : O(n) par recherche Tableau trié par id + recherche dichotomique : O(log n), mais insertion en O(n).

Choix 3 — Affichage par poids, valeur et priorité

Problème	Afficher les colis triés selon trois critères, avec jusqu'à 100 000 colis.
Solution	Tri fusion écrit à la main, une fonction par critère. On trie une copie des colis : le tableau d'origine ne bouge pas.
Temps	O(n log n)

Espace	O(n)
Alternative	Tri par insertion ou à bulles : O(n²)
Justification	Garantie O(n log n) même dans le pire cas.

3. Préparation du camion : réponses aux questions

Pourquoi avez-vous choisi cette stratégie ?

n est le nombre de colis dans la collection et cap est la capacité du camion en kg.

Le problème a les deux propriétés qui rendent la programmation dynamique adaptée.

1. Sous-structure optimale. Pour chaque colis, il n'y a que deux décisions : le charger ou non.
2. Sous-problèmes qui se répètent.

La solution est-elle toujours optimale ?

Oui. Pour chaque colis et chaque capacité, le tableau compare les deux seules possibilités (prendre ou laisser le colis) et garde la meilleure.

Complexité temporelle : O(n × cap).

Complexité spatiale : O(n × cap)

Autre approche possible ?
L'algorithme glouton trie les colis par rapport valeur/poids décroissant, c'est-à-dire du plus rentable par kilo au moins rentable. Il les parcourt ensuite dans cet ordre et charge chaque colis qui rentre encore dans le camion.

En revanche, il ne garantit pas la meilleure valeur. Chaque décision est définitive, et il ne regarde jamais si la place qu'il laisse pourra être bien remplie.

4. Recherche et performances

Opération 1 — Recherche d'un colis

Complexité : O(1) pour le meilleur cas et O(n) pour le pire cas.
Évolution : le temps reste quasi constant quand n est multiplié par 1 000.
Adapté ? Oui : c'est l'opération la plus utilisée (« l'entreprise recherche régulièrement des colis ») et elle ne ralentit pas.
Amélioration possible : Si les identifiants sont des entiers à peu près consécutifs (par exemple de 1 à 100 000), on peut supprimer le hachage. On utilise alors un tableau position[id] dont la case numéro id contient directement la position du colis.

Opération 2 — Tri des colis

Complexité : O(n log n).
Évolution : quand le nombre de colis est multiplié par 10, le temps du tri est multiplié par un peu plus de 10. C'est normal pour un tri fusion : chaque colis supplémentaire coûte un tout petit peu plus cher que le précédent.

Adapté ? Oui rapide même pour une collection assez grande..

5. Questions de synthèse

Quel est selon vous le choix algorithmique le plus important réalisé dans votre projet ? Pourquoi ?

La programmation dynamique pour le camion : c'est le seul point où un choix différent change le résultat et pas seulement le temps.

Quelle partie de votre programme risque de devenir problématique avec plusieurs millions de colis ?

La préparation du camion, à cause de sa mémoire O(n × cap). Les tris deviendraient aussi lourds.

Avez-vous privilégié à un moment la vitesse au détriment de la mémoire, ou inversement ?

Plutôt la vitesse

Une autre stratégie aurait-elle pu produire un meilleur résultat pour la préparation du camion ?

Non pour un critère donné : la programmation dynamique trouve déjà le meilleur score possible. Une autre règle de priorité donnerait un autre chargement: tout dépend de ce que l'entreprise veut privilégier.

Si vous deviez refaire le projet avec l'autre langage proposé (C ou Python), changeriez-vous vos choix algorithmiques ? Pourquoi ?

Les algorithmes resteraient les mêmes, car les complexités ne dépendent pas du langage.